

ABC 279 补题报告

C题

题目的主要意思其实是字符串的存储与查询，很容易想到用Trie树，但存在一个问题是读取时是横向读取，但我们需要存储的是列，所以不能边读取边存，数据范围不能够直接开二维数组，所以用string存但注意到两次存储的量是相等的，所以只需要存储第二次的记录从+改为-，只要存在负数就一定不成立

```
const int N=1e6+10;
typedef long long LL;
int n,m;
string c[N];
int son[N][26],idx,cnt[N];
bool flag=true;
void insert(char str[])
{
    int p=0;
    for(int i=0;str[i];i++)
    {
        int u=str[i]-'a';
        if(!son[p][u]) son[p][u]=++idx;
        p=son[p][u];
    }
    cnt[p]++;
}
void check(char str[])
{
    int p=0;
    for(int i=0;str[i];i++)
    {
        int u=str[i]-'a';
        if(!son[p][u]) son[p][u]=++idx;
        p=son[p][u];
    }
    cnt[p]--;
    if(cnt[p]<0) flag=false;
}
int main()
{
    scanf("%d%d",&n,&m);
    for(int i=0;i<n;i++)
    {
        cin>>c[i];
    }
    for(int i=0;i<m;i++)
    {
        char q[N];
        for(int j=0;j<n;j++)
        {
            if(c[j][i]=='#') q[j]='a';
        }
    }
}
```

```

        if(c[j][i]=='.') q[j]='b';
    }
    insert(q);
}
for(int i=0;i<n;i++)
{
    cin>>c[i];
}
for(int i=0;i<m;i++)
{
    char q[N];
    for(int j=0;j<n;j++)
    {
        if(c[j][i]=='#') q[j]='a';
        if(c[j][i]=='.') q[j]='b';
    }
    check(q);
}
if(flag) puts("Yes");
else puts("No");
return 0;
}

```

D题

主要还是考察浮点数运算，注意到要取整数，对原式子求导或者不等式找出该点附近的最小值即可，为了保险起见，前后遍历10个数，并且强制中间转换一下。

```

int main()
{
    long double a,b;
    cin>>a>>b;
    long double x=pow(a*1.0/(2*b),2.0/3)-1;
    long long t=x;
    long double res=a;
    for(long long i=t-10;i<=t+10;i++)
    {
        res=min(res,(long double) a/sqrt((long double)i+1)+(long double) i*b);
    }
    printf("%.1011f",res);
    return 0;
}

```

E题

从数据范围来看肯定不能暴力，可以先全部交换一边，b数组表示交换，c数组表示原来的顺序，用p数组存储b交换后的位置，最后遍历时只需要找到p[1]的位置，再恢复现场即可。

```

const int N=2e5+10;
int a[N],b[N],c[N],p[N];
int main()
{
    int n,m;

```

```

scanf("%d%d",&n,&m);
for(int i=1;i<=m;i++) scanf("%d",&a[i]);
for(int i=1;i<=n;i++) c[i]=b[i]=i;

    for(int j=1;j<=m;j++)
    {
        swap(b[a[j]],b[a[j]+1]);
    }
for(int i=1;i<=n;i++)
{
    p[b[i]]=i;
}
for(int i=1;i<=m;i++)
{
    swap(p[c[a[i]]],p[c[a[i]+1]]);
    printf("%d\n",p[1]);
    swap(p[c[a[i]]],p[c[a[i]+1]]);
    swap(c[a[i]],c[a[i] + 1]);
}
return 0;
}

```

自己附加一题 [VJ 第一周训练C题 跳石头](#) (二分答案法)

因为答案一定存在一个整数的长度满足当前条件，所以一定能二分找出来满足条件的长度。

```

#include <bits/stdc++.h>
using namespace std;
const int N=50010;
int a[N];
int d,n,m;
bool check(int x)
{
    int sum=0;
    int p=0;
    for(int i=1;i<=n;i++)
    {
        if(a[i]-a[p]<x)
        {
            sum++;
        }
        else
        {
            p=i;
            continue;
        }
    }
    if(sum>m) return false;
    else return true;
}

```

```

}
int main()
{
    scanf("%d%d%d",&d,&n,&m);
    for(int i=1;i<=n;i++)
    {
        scanf("%d",&a[i]);
    }
    a[n+1]=d;
    int l,r,mid,res=0;
    l=1;
    r=d;
    while (l<=r)
    {
        mid=(l+r)/2;
        if(check(mid))
        {
            res=mid;
            l=mid+1;
        }
        else r=mid-1;
    }
    printf("%d\n",res);
    return 0;
}

```

ps： 错误代码

(原先想法是用小根堆查找出当前最小值，将其向前或向后合并，再插入堆中，还用了l, r数组来维护左右未被合并的距离，但实际上并不知道是向左还是向右合并，没有一定的规律，所以不能这样做)

```

#include <bits/stdc++.h>
using namespace std;
const int N=1e6+10;
int q,n,m;
typedef pair<int,int> PII;

int a[N],b[N],l[N],r[N];
struct cmp
{
    bool operator ()(PII a,PII b)
    {
        return a.second>b.second;
    }
};
int main()
{
    priority_queue<PII,vector<PII>,cmp >heap;
    scanf("%d%d%d",&q,&n,&m);
    for(int i=1;i<=n;i++)
    {
        scanf("%d",&a[i]);
    }
    if(n==m)

```

```

{
    printf("%d\n",q);
    return 0;
}
a[n+1]=q;
for(int i=n;i>=1;i--)
{
    b[i]=a[i+1]-a[i];
    heap.push({i,b[i]});
}

for(int i=1;i<=m;i++)
{
    PII t=heap.top();
    heap.pop();
    int k=t.first;

    if(b[k]!=-1)
    {
        if(b[k+r[k]+1]>b[k-1-l[k]])
        {
            b[k]+=b[k+r[k]+1];
            l[k+1+r[k]+1+r[k+r[k]+1]]+=r[k]+1;
            r[k]+=r[k+r[k]+1]+1;
            b[k+r[k]+1]=-1;
            heap.push({k,b[k]});
        }
        else
        {
            b[k-1-l[k]]+=b[k];
            l[k+1+r[k]]=l[k]+1;
            r[k-1-l[k]]+=r[k]+1;
            b[k]=-1;
            heap.push({k-1-l[k],b[k-1-l[k]]});
        }
    }
    else m--;

}
cout<<heap.top().second<<endl;
// for(int i=1;i<=n;i++)printf("%d\n",b[i]);
return 0;
}

```